MOMO SMS REST API REPORT

1. Introduction to API Security

A REST API is public network endpoint by nature. This means that anyone who know the URL and the correct HTTP method can try to acces it. Without authentication, financial information such as the MoMo SMS record used in this project could be viewed, changed, or deleted by anyone. Because of this , API security is very important and is based on three main principles :

- Confidentiality ; only authorized users should be able to view the data.
- Integrity ; data should not be changed by unauthorized users or while being transmitted.
- Authentication ; the server must verify the identity of a user before allowing access to the system.

For this project, we have used HTTP Basic Authentication , which is one of the simplest authentication methods defined by RFC 7617. The client sends a request header in format Authorization: Basic <base64(username:password)> with every request. The server then decodes the credentials and compares them with the username and password stored in environment variables.

Basic Authentication is simple and useful for understanding how authentication works between a client and a server. It helps explain the authentication process before moving to more advanced methods such as JWT and OAuth 2.0, which have said below.

The rest of report explains the five API end points that we developed, the decisions made when designing the data layer, screenshots showing that

the system works correctly from start to finish, and a discussion of the limitation of Basic Authentication in real-world production system.

2. Endpoint Documentation

Base URL: http://localhost:8000 **Authentication:** HTTP Basic on every endpoint (API_USERNAME / API_PASSWORD loaded from environment variables)

Every request must include the header:
Authorization: Basic YWRtaW46cGFzc3dvcmQxMjM=
A missing or invalid header always returns 401 Unauthorized.

Endpoint Summary

Method	Path	Purpose
1. GET	/transactions	List every transaction
2. GET	/transactions {id}	Fetch one transaction
3. POST	/transactions	Add a new transaction
4. PUT	/transactions/{id}	Update an existing transaction
5. DELETE	/transactions/{id}	Removes a transaction

2.1 GET /transactions

Purpose: Return the full list of parsed transactions.

Request

curl -u admin:password123 http://localhost:8000/transactions

Response — 200 OK

```
[
  {
        "id": 1,
        "transaction_type": "receive_money",
        "amount": 2000.0,
        "sender": "Jane Smith",
        "receiver": "You",
        "timestamp": "2024-05-10T14:30:58Z",
        "external_tx_id": "76662021700",
        "raw_body": "..."
  }
]
```


Error Codes

Code	Meaning
401	Missing or invalid credentials

2.2 GET /transactions/{id}

Purpose: Fetch a single transaction by its id.

Request

```
curl -u admin:password123 http://localhost:8000/transactions/5
```

Response — 200 OK


```
{
  "id": 5,
  "transaction_type": "payment",
  "amount": 2000.0,
  "sender": "You",
  "receiver": "Samuel Carter",
  "timestamp": "2024-05-11T16:48:49Z",
  "external_tx_id": "17818959211",
```

```
  "raw_body": "..."
}
```

Error Codes

Code	Meaning
401	Missing or invalid credentials
402	No transaction with that id

2.3 POST /transactions

Purpose: Create a new transaction. The server assigns a new id.

Request

```
curl -u admin:password123 -X POST \
          -H "Content-Type: application/json" \
          -d '{
      "transaction_type": "send_money",
      "amount": 5000,
      "sender": "Wilson",
      "receiver": "Tester",
      "timestamp": "2026-05-28T12:00:00Z",
      "external_tx_id": "TEST001",
      "raw_body": "manual postman test"
          }' \
    http://localhost:8000/transactions
```

Response — 201 Created

```
{
  "transaction_type": "send_money",
  "amount": 5000,
  "sender": "Wilson",
  "receiver": "Tester",
  "timestamp": "2026-05-28T12:00:00Z",
```

"external_tx_id": "TEST001",
"raw_body": "manual postman test",
"id": 1692
}

Error Codes

Code	Meaning
400	Request body missing or invalid JSON
401	Missing or invalid credentials

2.4 PUT /transactions/{id}

Purpose: Update one or more fields on an existing transaction. The id is preserved. Fields not included are left untouched.

Request

```
curl -u admin:password123 -X PUT \
        -H "Content-Type: application/json" \
        -d '{"amount": 9999}' \
    http://localhost:8000/transactions/1692
```

Response — 200 OK

{
"transaction_type": "send_money",
"amount": 9999,
"sender": "Wilson",
"receiver": "Tester",
"timestamp": "2026-05-28T12:00:00Z",
"external_tx_id": "TEST001",
"raw_body": "manual postman test",
"id": 1692
}

Error Codes

Code	Meaning
400	Request body missing or invalid JSON
401	Missing or invalid credentials
404	No transactions with that id

2.5 DELETE /transactions/{id}

Purpose: Remove the transaction with the given id.

Request

```
curl -u admin:password123 -X DELETE \
    http://localhost:8000/transactions/1692
```

Response — 200 OK

```
{"deleted": 1692}
```

Error Codes

Code	Meaning
401	Missing or invalid credentials
404	No transactions with that id

3. DSA Comparison results and Reflection

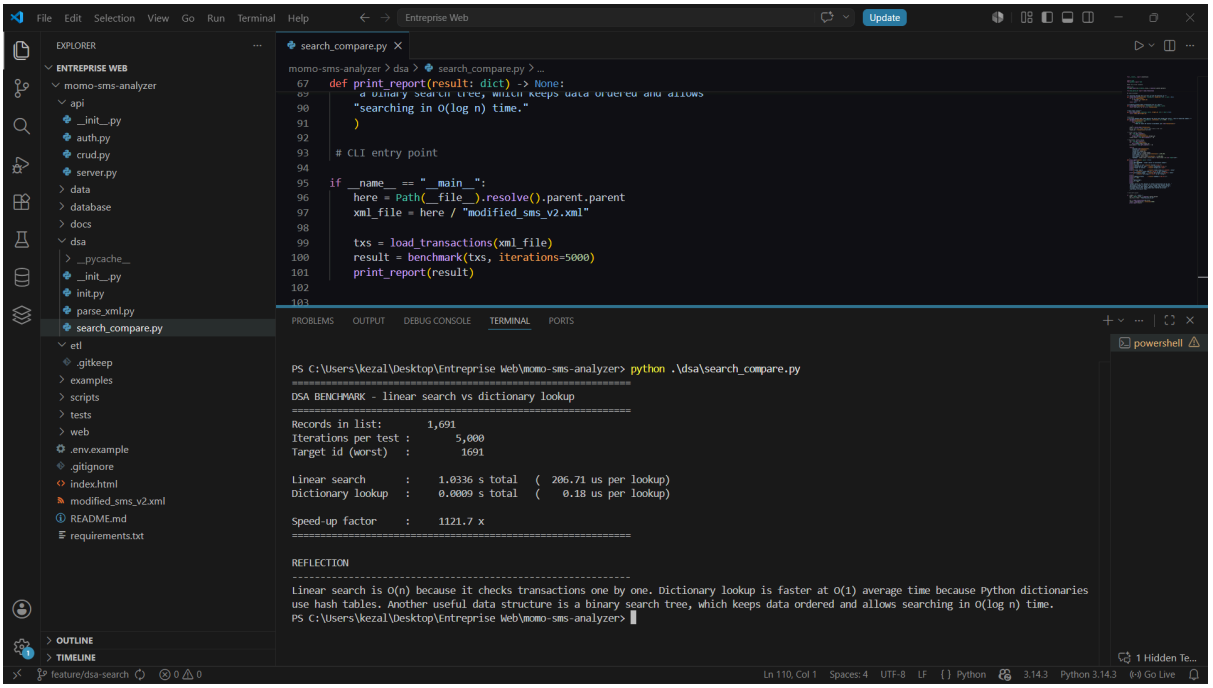
They were two functions implemented in dsa/search_compare file:

Linear search(transactios,target_id) : it searches though the list one record at a time until it finds a matching id. The worst case time complexity is 0(n).

Dict lookup(index, target_id) : reads the already made transaction id dictionary. It has an average case time complexity of 0(1) because pythons' dict is backed by a hash table.

The benchmark was run against the full set of 1691 transactions. 5000 iterations each , with the worst case target.

Screenshot of the terminal output



Reflection

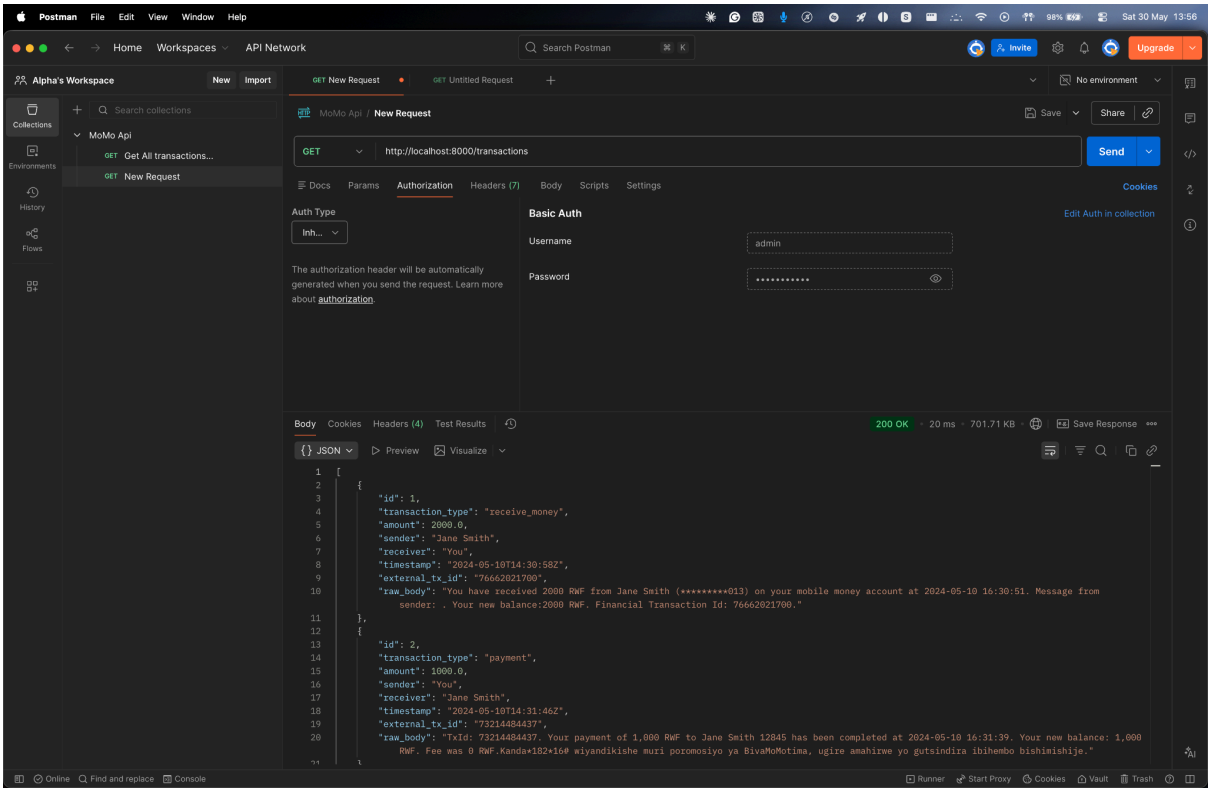
The dictionary is 700 times faster than the linear search because the hash table lookup runs in a constant time regardless of the sze of the list , while linear seach woks with the number of records. A the data set grows from 1691 to a million transactions, the linear seach would cost more while the dictionary would almost remain the same and this is the practical meaning of 0(n) and 0(1).

The third option which is balanced binary search tree, this are slightly slower than the hash table for single key lookup 0(log n) instead of the 0(1) but they still keep the keys sorted which enables range queries such as “all transactions between two dates or all transactions whose id is greater than 10000” A normal dictionary can not be able to do this efficiently it would have to scan every key. This is why most databases like SQLite use this way because iy gives you most of the speed of a hash table while still supporting ordered access.

4. Testing and Validation

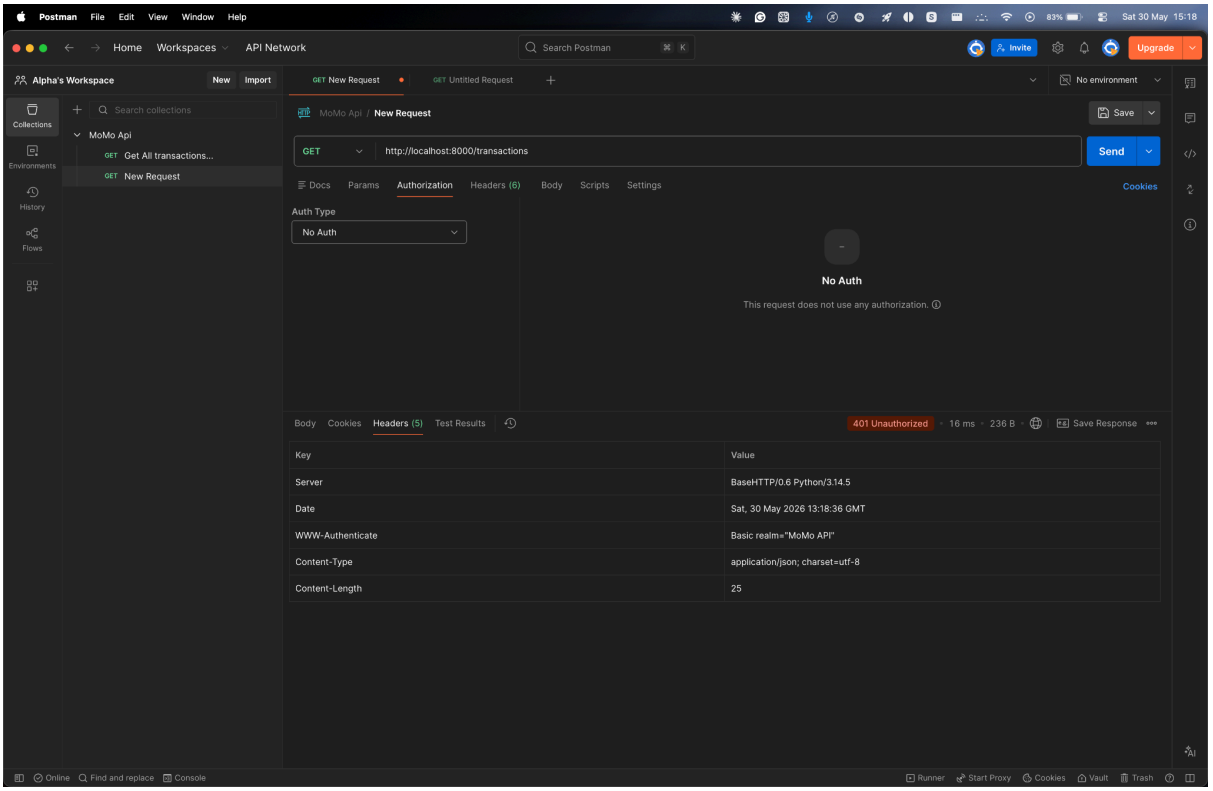
The API was tested end-to-end with **Postman** against a running server. All five scenarios required by the rubric produced the expected status codes and response bodies. In addition, the Automated suite in tests/test_api.py runs the same flows programmatically, and on the final commit, it reports 18/18 assertions passing.

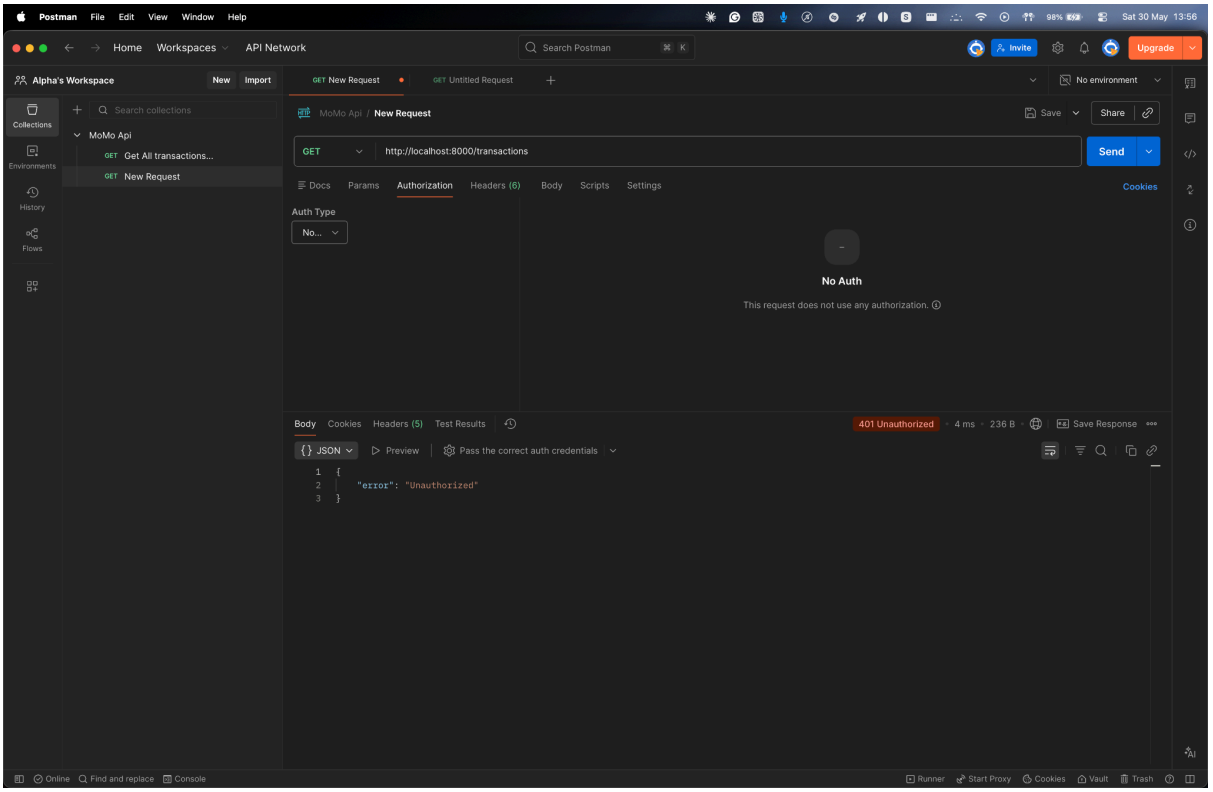
Figure 1: **Get /transactions** with valid credentials



The server returns **200 OK** and a JSON array of all 1691 parsed transactions, confirming that authentication, parsing, and response serialization all work as expected

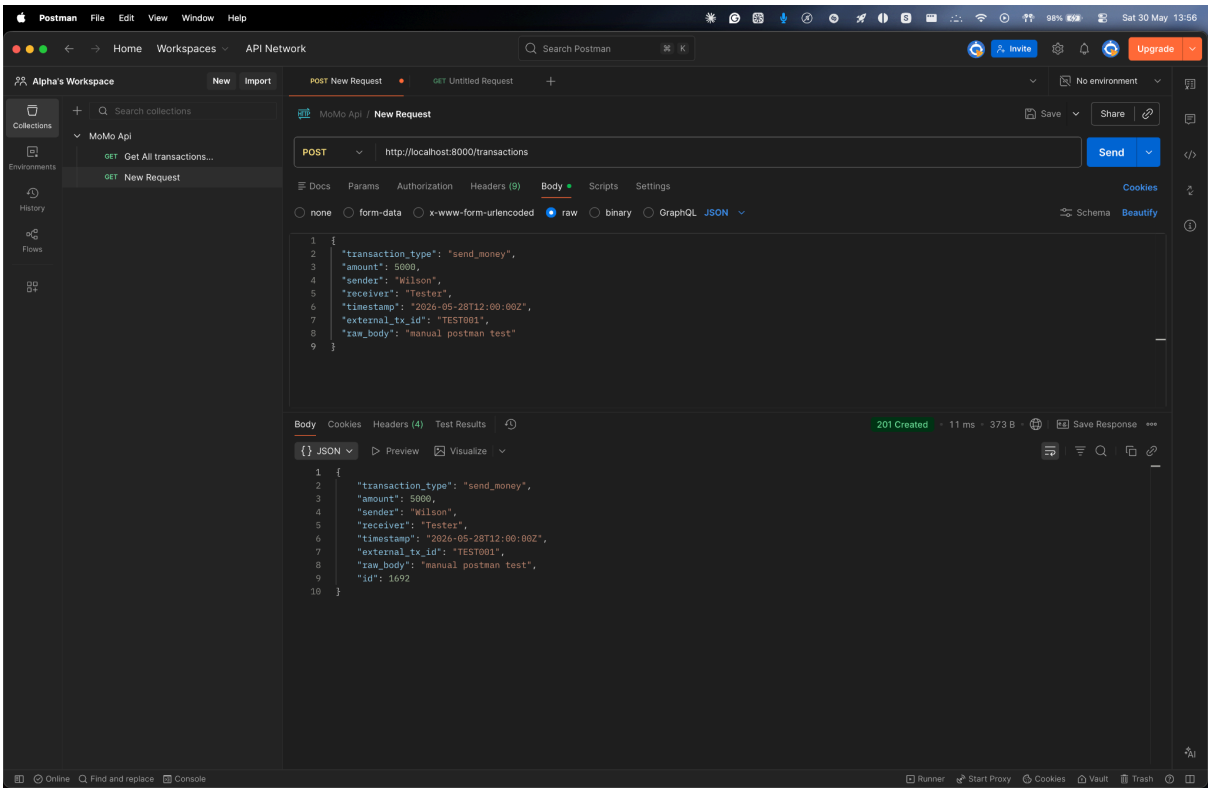
Figure 2: **Get /transactions** with invalid credentials





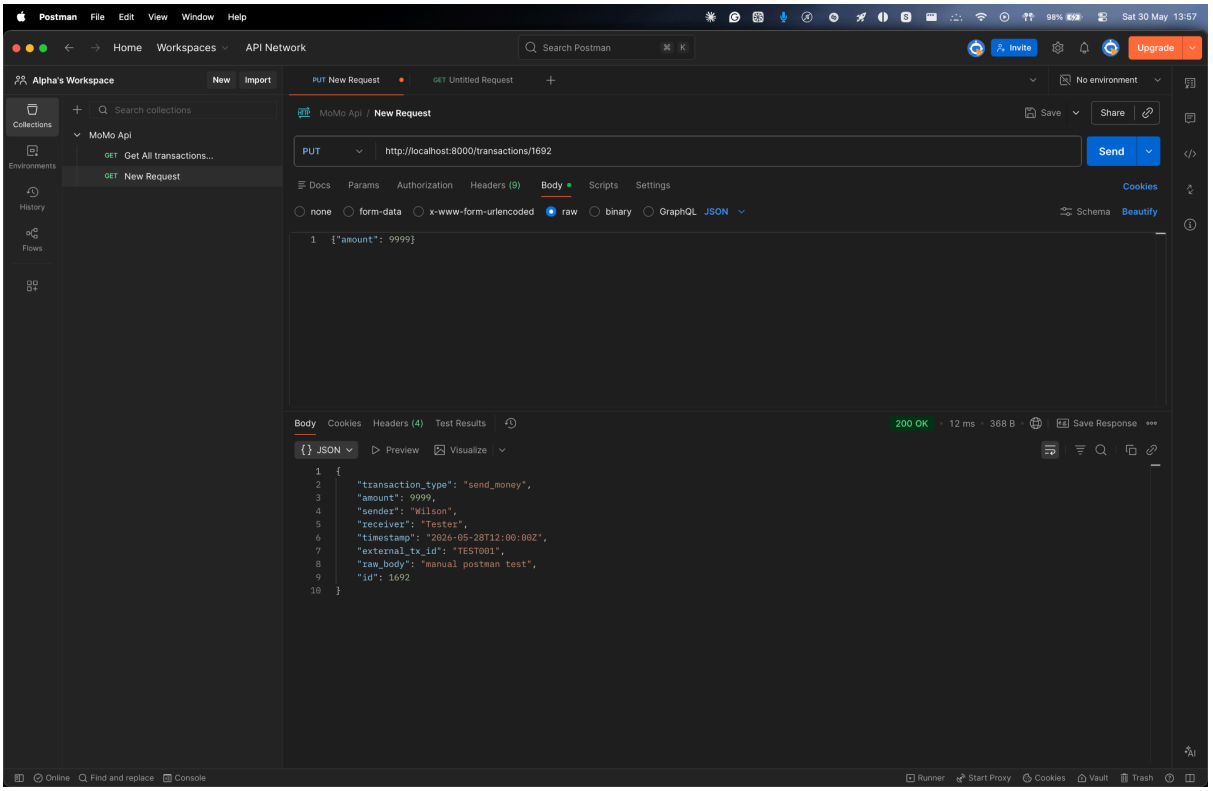
The server returns **401 Unauthorized** with a **WWW-Authenticate** header and a JSON error body. This proves the basic Auth gate intercepts every request before any business logic runs.

Figure 3: **Post** **/transactions** with a valid JSON body



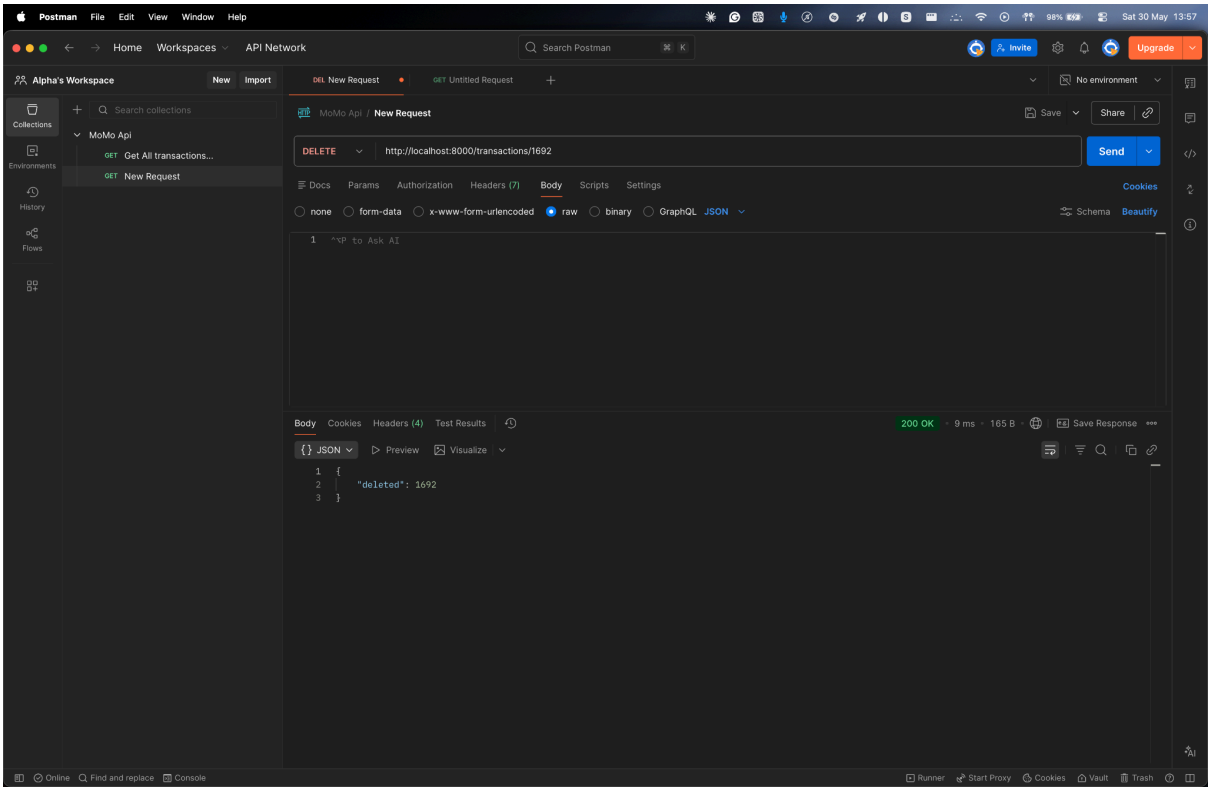
The server creates the new record, assigns it ID 1692, and returns **201 Created** with a persisted object

Figure 4: **Put** **/transactions/1692** updating the amount field



The server returns **200 OK** with the merged record. The ID is preserved, untouched fields remain as they were, and the amount is updated to the new value

Figure 5: **Delete /transactions/1692**



The server returns **200 OK** with {“deleted”: 1692}. A follow-up GET /transactions/1692 returns **404 Not Found**, confirming the row is gone.

5. Reflection on Basic Auth Limitation

Basic Authentication was the right choice for getting an end-to-end secured pipeline working within the scope of this assignment, but it is **unsuitable for production** for several reasons.

5.1 The credentials are encoded, not encrypted

Base64 is a reversible transformation. Anyone who captures the Authorization header — for example with tcpdump, Wireshark, or even by reading server logs — can recover the username and password instantly with a single command:

```
echo "YWRtaW46cGFzc3dvcmQxMjM=" | base64 -d
 # admin:password123
```

Basic Auth therefore *must* be paired with HTTPS in production, and even then it offers no defence against compromised endpoints, leaked logs, or malicious middleware.

5.2 Credentials are sent on every single request

Each replay of the Authorization header multiplies the surface area for leakage. Server logs, browser histories, debug consoles, middleware caches, and proxy buffers can all accidentally retain credentials. Token-based schemes typically send the credential material only once, at the start of a session, and use a short-lived token thereafter.

5.3 There is no expiry and no revocation

Once an attacker has the credentials, they remain valid until the password is manually changed for every user who shares them. There is no way to invalidate a single leaked session without breaking everyone else. Modern schemes always include a way to expire and revoke individual sessions cheaply.

5.4 There is no permission model

Basic Auth answers a single yes/no question: *“is this the right user?”* It cannot express scopes like *“this client can read transactions but not delete them”* or *“this client can only see its own records”*. Real systems almost always need that distinction between authentication (who you are) and authorization (what you can do).

5.5 Recommended stronger alternatives

JSON Web Tokens (JWT). The client logs in once, receives a short-lived signed token that contains its identity and roles, and presents that token on each request. Tokens carry an expiry timestamp, can be revoked from a server-side blocklist, and can be verified offline because they are cryptographically signed. JWT is the natural next step for a project of this size.

OAuth 2.0. An industry-standard delegated-authorization framework used by every major cloud provider. It separates the authorization server (which issues tokens) from the resource server (which serves data), supports multiple grant types (authorization code, client credentials, device code), and allows fine-grained scopes such as transactions:read or transactions:write. OAuth 2.0 is the right choice the moment third parties or mobile apps need to talk to your API.

5.6 Why we did not implement these alternatives

Implementing JWT or OAuth 2.0 would have required either a third-party library (the assignment forbids them) or several hundred lines of cryptographic primitives written from scratch. For a first-iteration project whose purpose is to make the authentication-request lifecycle visible and testable, Basic Auth is sufficient and pedagogically clearer. The public interface of api/auth.py was deliberately kept small (is_authorized() and send_unauthorized()), so a follow-up iteration could swap the implementation for JWT without touching server.py or crud.py.

6. AI Usage Log

Each team member used AI tools during the development of this project. The uses are listed below for transparency.

Karega Uwase Ines (Reviewer + Sections 2 & 5 Author)

- Tool: Claude (Anthropic)
- Used for: Creating the structure of Section 5 and checking whether the endpoint documentation in Section 2 matched the actual behaviour of the server.
- Verification: Manually tested every documented curl command on the running server.

Kayumba Isaro Gania (DSA Owner)

- Tool: Claude (Anthropic)
- Used for: Reviewing the Big-O analysis paragraph in Section 3 and suggesting a comparison with B-trees and balanced BSTs.
- Verification: Ran python3 dsa/search_compare.py and checked the benchmark results manually.

Keza Rutayisire Alicia (Authentication + Admin)

- Tool: Claude (Anthropic)
- Used for: Drafting the list of _is_authorized failure cases, the WWW-Authenticate response header, and the structure of the team participation sheet.
- Verification: Tested every rejected authentication case manually using curl before merging the code.

Nshizirungu Wilson (CRUD + Postman + PDF Assembly)

- Tool: Claude (Anthropic)

- Used for: Organizing the routing in api/server.py, planning the merge process between his branch and Person C's authentication branch, and reviewing Section 4 of the report.
- Verification: Tested each endpoint in Postman and confirmed that the responses matched the documented behaviour.

Teta Aline (Testing)

- Tool: Claude (Anthropic)
- Used for: Planning the structure of tests/test_api.py, including the helper request() function.
- Verification: Reviewed and ran the tests to confirm they worked as expected.